

Module 5: Iterative Control Statements (Loops) in PL/SQL

Duration

6–8 Hours (Theory + Laboratory Practice)

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of iteration (looping).
 - Explain the need for loops in programming.
 - Use the Simple LOOP statement.
 - Implement WHILE LOOP and FOR LOOP.
 - Use nested loops to solve complex problems.
 - Control loop execution using EXIT, EXIT WHEN, and CONTINUE.
 - Develop programs involving repetitive tasks such as calculations, pattern generation, and table generation.
-

5.1 Introduction to Iterative Statements

What is Iteration?

Iteration is the process of executing a block of statements repeatedly until a specified condition becomes FALSE.

In programming, many tasks involve repetition. Instead of writing the same statements multiple times, loops allow us to execute them efficiently.

Real-Life Examples

Imagine a teacher wants to print the attendance numbers from 1 to 60.

Without a loop:

Roll No: 1

Roll No: 2

Roll No: 3

...

Roll No: 60

The teacher would have to write 60 statements manually.

With a loop, a single block of code can generate all 60 numbers automatically.

Why Do We Need Loops?

Consider displaying numbers from 1 to 100.

Without loops:

```
DBMS_OUTPUT.PUT_LINE(1);
```

```
DBMS_OUTPUT.PUT_LINE(2);
```

```
DBMS_OUTPUT.PUT_LINE(3);
```

...

```
DBMS_OUTPUT.PUT_LINE(100);
```

This approach is inefficient.

Using loops:

```
FOR i IN 1..100 LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(i);
```

```
END LOOP;
```

Advantages:

- Reduces code length.
 - Improves readability.
 - Reduces programming errors.
 - Makes maintenance easier.
-

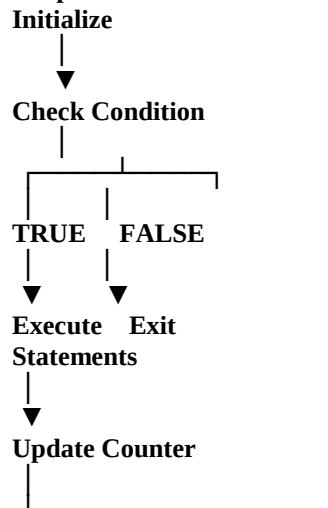
Types of Loops in PL/SQL

PL/SQL provides three types of loops.

Loop	Description
------	-------------

Loop	Description
Simple LOOP	Executes until EXIT condition is encountered
WHILE LOOP	Executes while a condition is TRUE
FOR LOOP	Executes for a fixed number of iterations

Loop Execution Flow



5.2 Simple LOOP

What is Simple LOOP?

The Simple LOOP executes continuously until an EXIT statement is encountered. It does not check any condition before starting execution.

Syntax

LOOP

statements;

EXIT WHEN condition;

END LOOP;

Example 1: Print Numbers from 1 to 10

SET SERVEROUTPUT ON;

DECLARE

i NUMBER := 1;

BEGIN

LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

EXIT WHEN i > 10;

```
END LOOP;
```

```
END;
```

```
/
```

Output

```
1
2
3
4
5
6
7
8
9
10
```

Example 2: Sum of First 10 Numbers

```
DECLARE
```

```
    i NUMBER := 1;
```

```
    total NUMBER := 0;
```

```
BEGIN
```

```
    LOOP
```

```
        total := total + i;
```

```
        i := i + 1;
```

```
        EXIT WHEN i > 10;
```

```
    END LOOP;
```

```
    DBMS_OUTPUT.PUT_LINE('Total = ' || total);
```

```
END;
```

```
/
```

Output

```
Total = 55
```

5.3 WHILE LOOP

What is WHILE LOOP?

A WHILE LOOP executes repeatedly as long as the specified condition remains TRUE.

If the condition is FALSE initially, the loop body is never executed.

Syntax

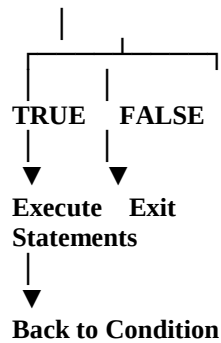
WHILE condition LOOP

```
    statements;
```

```
END LOOP;
```

Flow Diagram

Check Condition



Example 1: Print Numbers 1 to 10
DECLARE

i NUMBER := 1;

BEGIN

WHILE i <= 10 LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

END LOOP;

END;

/

Example 2: Multiplication Table
DECLARE

i NUMBER := 1;

BEGIN

WHILE i <= 10 LOOP

DBMS_OUTPUT.PUT_LINE('5 x ' || i || ' = ' || (5*i));

i := i + 1;

END LOOP;

END;

/

5.4 FOR LOOP

What is FOR LOOP?

The FOR LOOP is used when the number of iterations is known in advance.

Oracle automatically initializes, increments, and terminates the loop variable.

Syntax

FOR variable IN lower..upper LOOP

```
statements;  
END LOOP;
```

Example 1: Print Numbers
BEGIN

```
FOR i IN 1..10 LOOP  
  
    DBMS_OUTPUT.PUT_LINE(i);  
  
END LOOP;  
  
END;  
/
```

Example 2: Reverse Loop
BEGIN

```
FOR i IN REVERSE 1..10 LOOP  
  
    DBMS_OUTPUT.PUT_LINE(i);  
  
END LOOP;  
  
END;  
/
```

Output
10

9
8
7
6
5
4
3
2
1

Comparison of Loops

Feature	Simple LOOP	WHILE LOOP	FOR LOOP
Condition Check	Inside loop	Before execution	Automatic
Counter	Manual	Manual	Automatic
Best Used When	Unknown iterations	Condition-based	Fixed iterations

5.5 Nested Loops

What is Nested Loop?

A loop inside another loop is called a Nested Loop.

Used for:

- **Pattern printing**
 - **Matrix operations**
 - **Multiplication tables**
-

Example: Multiplication Table

```

BEGIN

  FOR i IN 1..5 LOOP

    FOR j IN 1..10 LOOP

      DBMS_OUTPUT.PUT_LINE(i || ' x ' || j || ' = ' || i*j);

    END LOOP;

    DBMS_OUTPUT.PUT_LINE('-----');

  END LOOP;

END;
/

```

5.6 EXIT Statement

What is EXIT?

EXIT immediately terminates the loop.

Syntax

EXIT;

Example

DECLARE

 i NUMBER := 1;

BEGIN

 LOOP

 DBMS_OUTPUT.PUT_LINE(i);

 IF i = 5 THEN

 EXIT;

 END IF;

 i := i + 1;

 END LOOP;

END;

/

Output

1
2
3
4
5

5.7 EXIT WHEN

Instead of writing IF...EXIT, Oracle provides EXIT WHEN.

Example

DECLARE

i NUMBER := 1;

BEGIN

LOOP

DBMS_OUTPUT.PUT_LINE(i);

i := i + 1;

EXIT WHEN i > 5;

END LOOP;

END;

/

5.8 CONTINUE Statement

What is CONTINUE?

CONTINUE skips the remaining statements of the current iteration and proceeds to the next iteration.

Example

BEGIN

FOR i IN 1..10 LOOP

CONTINUE WHEN MOD(i,2)=0;

DBMS_OUTPUT.PUT_LINE(i);

END LOOP;

END;

/

Output

1

3

5

7

9

Practical Programs

Program 1: Factorial

DECLARE

n NUMBER := 5;

fact NUMBER := 1;

BEGIN

FOR i IN 1..n LOOP

```
        fact := fact * i;

    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Factorial = ' || fact);

END;
/
```

Program 2: Fibonacci Series

```
DECLARE

    a NUMBER := 0;

    b NUMBER := 1;

    c NUMBER;

BEGIN

    DBMS_OUTPUT.PUT_LINE(a);

    DBMS_OUTPUT.PUT_LINE(b);

    FOR i IN 3..10 LOOP

        c := a + b;

        DBMS_OUTPUT.PUT_LINE(c);

        a := b;

        b := c;

    END LOOP;

END;
/
```

Program 3: Prime Number

```
DECLARE

    n NUMBER := 17;

    count_factor NUMBER := 0;

BEGIN

    FOR i IN 1..n LOOP

        IF MOD(n,i)=0 THEN

            count_factor := count_factor + 1;

        END IF;

    END LOOP;

END;
```



```
END LOOP;

IF count_factor=2 THEN

    DBMS_OUTPUT.PUT_LINE('Prime Number');

ELSE

    DBMS_OUTPUT.PUT_LINE('Not Prime');

END IF;

END;
/
```

Program 4: Reverse a Number

```
DECLARE

    num NUMBER := 12345;

    rev NUMBER := 0;

BEGIN

    WHILE num > 0 LOOP

        rev := rev * 10 + MOD(num,10);

        num := TRUNC(num/10);

    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Reverse = ' || rev);

END;
/
```

Real-World Applications

Loops are widely used in:

- Payroll Processing
 - Monthly Salary Generation
 - Student Result Processing
 - Bank Interest Calculations
 - Attendance Reports
 - Invoice Generation
 - Batch Data Updates
 - Data Migration Scripts
 - Report Generation
 - Email Notifications
-

Common Errors by Beginners

- Forgetting to increment the loop counter, leading to an infinite loop.
- Using an incorrect loop termination condition.
- Modifying the loop index variable inside a FOR LOOP (not allowed).

- Omitting EXIT or EXIT WHEN in a Simple LOOP.
 - Writing END instead of END LOOP.
-

Summary

This module introduced iterative control statements in PL/SQL, enabling students to automate repetitive tasks. We explored the Simple LOOP, WHILE LOOP, and FOR LOOP, learned how to terminate or skip iterations using EXIT, EXIT WHEN, and CONTINUE, and implemented practical programs such as factorial, Fibonacci series, prime number checking, and number reversal. Mastering loops is essential for developing efficient PL/SQL applications and prepares students for advanced topics like cursors and collections.

Laboratory Exercises

1. Print numbers from 1 to 100 using all three loop types.
 2. Display the multiplication table of any number.
 3. Calculate the sum of the first N natural numbers.
 4. Find the factorial of a given number.
 5. Generate the Fibonacci series up to N terms.
 6. Check whether a number is prime.
 7. Reverse a given number.
 8. Count the number of digits in an integer.
 9. Print all even and odd numbers between 1 and 100.
 10. Generate a pyramid or triangle star pattern using nested loops.
 11. Calculate the sum of digits of a number.
 12. Display the multiplication tables from 1 to 10 using nested loops.
-

Viva/Interview Questions

1. What is iteration in PL/SQL?
2. Differentiate between Simple LOOP, WHILE LOOP, and FOR LOOP.
3. When should a FOR LOOP be preferred over a WHILE LOOP?
4. What is the purpose of the EXIT statement?
5. Explain EXIT WHEN with an example.
6. What is the function of the CONTINUE statement?
7. Can the loop index variable be modified inside a FOR LOOP?
8. What is an infinite loop, and how can it be avoided?
9. Explain the concept of nested loops.
10. List some real-world applications where loops are commonly used in PL/SQL.